DevOps et intégration continue

Informatique — Licence 3 (L3)

Cours INF-L3-12

Institut Africain de Management (IAM) — Bamako, Mali
Bibliothèque Universitaire — Bronbah Tech

À propos de ce cours
Livrer vite et sans casser : culture DevOps, intégration continue, stratégies de déploiement, conteneurisation Docker, supervision et infrastructure as code, jusqu'au pipeline complet d'une application réelle.

Document pédagogique original créé pour les étudiants de l'IAM : leçons détaillées, exemples commentés, travaux dirigés et pratiques, exercices corrigés, QCM d'évaluation et vidéos recommandées. Aucun contenu n'est reproduit d'ouvrages existants.

Sommaire

• Introduction à la matière
• Objectifs d'apprentissage
• Leçon 1 — Culture DevOps et cycle de livraison logicielle
• Leçon 2 — Intégration continue : automatiser tests et constructions
• Leçon 3 — Déploiement continu et stratégies de mise en production
• Leçon 4 — Conteneuriser une application avec Docker
• Leçon 5 — Supervision, journaux et alertes en production
• Leçon 6 — Infrastructure as Code : initiation aux principes
• Leçon 7 — Projet : mettre en place un pipeline complet de déploiement
• Travaux dirigés (TD)
• Travaux pratiques (TP)
• Exercices corrigés supplémentaires
• QCM d'évaluation
• Résumé — les notions importantes
• Vidéos associées aux leçons
• Sources et références

Comment utiliser ce cours : lis chaque leçon dans l'ordre, refais les exemples à la main, tente chaque exercice AVANT de lire son corrigé, puis termine par le QCM pour vérifier ce que tu as retenu.

Introduction à la matière

Tu sais déjà écrire une application (L2), la tester et la déployer dans le cloud (L3-05). Il reste un problème que rencontrent toutes les équipes : le passage du code à la production. Quand ce passage est manuel, il est lent, stressant et différent à chaque fois ; une fois sur deux, « ça marchait sur ma machine » devient « ça ne marche plus en production ». Le DevOps est la réponse des professionnels à ce problème : une culture de collaboration entre développeurs (Dev) et exploitants (Ops), soutenue par l'automatisation de tout ce qui peut l'être.

Ce cours construit, leçon après leçon, le pipeline complet d'une application fictive mais réaliste : BiblioAPI, l'API de la bibliothèque de l'IAM qui gère les livres et les emprunts. Tu écriras son pipeline d'intégration continue, ses stratégies de déploiement progressif, son Dockerfile, sa supervision et la description codée de son infrastructure. À la fin, un commit sur la branche principale suffira à tester, construire et déployer l'application de façon reproductible.

Aucun outil n'est imposé : les exemples utilisent une syntaxe de pipeline générique (proche de GitHub Actions et de GitLab CI) et des fichiers Docker standard, transposables à n'importe quelle plateforme. Ce qui compte, ce sont les principes : automatiser, tester tôt, déployer par petits pas, tout versionner — et ne jamais écrire un secret en clair dans un fichier du dépôt.

Objectifs d'apprentissage

À la fin de ce cours, tu seras capable de :

- Expliquer la culture DevOps et décrire les étapes du cycle de livraison logicielle.
- Écrire un pipeline d'intégration continue qui construit et teste l'application à chaque commit.
- Distinguer livraison continue et déploiement continu, et choisir une stratégie de mise en production (rolling, blue-green, canary).
- Conteneuriser une application avec un Dockerfile propre et l'orchestrer localement avec Docker Compose.
- Mettre en place une supervision utile : métriques, journaux structurés et alertes à seuils cohérents.
- Décrire une infrastructure en code déclaratif, versionné et révisable comme le code applicatif.
- Assembler un pipeline complet, du commit à la production, sans aucun secret en clair.

Leçon 1 — Culture DevOps et cycle de livraison logicielle

Dans l'organisation classique, deux équipes se succèdent sans se parler : les développeurs écrivent le code, puis le « lancent par-dessus le mur » aux exploitants, chargés de l'installer sur les serveurs. Chaque camp optimise son propre objectif : du changement rapide d'un côté, de la stabilité à tout prix de l'autre. Résultat : les mises en production sont rares, énormes et redoutées, et quand quelque chose casse, chacun renvoie la faute à l'autre camp. Le DevOps supprime ce mur : une seule équipe est responsable de l'application de l'idée jusqu'à l'exploitation — « tu construis, tu exploites ».

Concrètement, le cycle de livraison devient une boucle courte et répétée : planifier, coder, construire, tester, livrer, déployer, exploiter, superviser — puis recommencer, la supervision nourrissant la planification suivante. Chaque tour livre un petit changement plutôt qu'une montagne de nouveautés : un petit changement est facile à tester, facile à déployer et, en cas de problème, facile à identifier et à annuler. C'est l'inverse du « gros bang » semestriel, où l'on ne sait jamais quelle modification, parmi deux cents, a provoqué la panne.

On mesure la santé d'une organisation DevOps avec quatre indicateurs popularisés par les études DORA : la fréquence de déploiement (combien de mises en production par jour ou par semaine), le délai de mise en œuvre (temps entre un commit et sa présence en production), le temps moyen de restauration après incident, et le taux d'échec des changements. Les équipes performantes déploient plusieurs fois par jour avec un taux d'échec faible : la preuve que vitesse et stabilité ne s'opposent pas, à condition d'automatiser les tests et de déployer par petits pas.

L'essentiel
DevOps = une culture (une équipe responsable de bout en bout) + des pratiques (petits changements fréquents) + de l'automatisation (le pipeline). On juge une équipe à quatre indicateurs : fréquence de déploiement, délai commit → production, temps de restauration, taux d'échec des changements.

Exemple — Le cycle de livraison de BiblioAPI, et le avant/après

```
LE CYCLE DE LIVRAISON DE BIBLIOAPI (boucle DevOps)

1. Planifier : les demandes des bibliothécaires (tickets)
2. Coder : branches courtes, revue de code entre pairs
3. Construire : le pipeline assemble l'application (build)
4. Tester : tests automatisés à chaque commit
5. Livrer : une version candidate, prête pour la production
6. Déployer : mise en production progressive (canary)
7. Exploiter : l'application sert les étudiants
8. Superviser : métriques et journaux, retour à l'étape 1

AVANT (manuel) : déploiement le vendredi soir, 2 heures à la
main, résultat différent selon la personne présente.
APRÈS (DevOps) : déploiement en 6 minutes, identique à chaque
fois, retour arrière possible en 1 minute.
```

À retenir

- Le DevOps rapproche développement et exploitation dans une même responsabilité, au lieu de deux silos qui se renvoient la faute.
- Déployer souvent de petits changements est MOINS risqué que déployer rarement de gros changements.
- Les quatre indicateurs DORA (fréquence, délai, restauration, taux d'échec) mesurent à la fois la vitesse ET la stabilité.

Exercice

Une entreprise déploie son application une fois par trimestre, le dimanche, en arrêtant le service pendant 6 heures. À chaque fois, au moins un incident majeur survient et le retour arrière prend une journée. En utilisant les idées de cette leçon, liste quatre changements concrets qu'elle devrait adopter, et explique pour chacun quel indicateur DORA il améliorerait.

✔ Corrigé
1) Automatiser la construction et les tests à chaque commit (pipeline) : détecte les régressions en quelques minutes au lieu du dimanche de déploiement → améliore le taux d'échec des changements. 2) Découper les livraisons : déployer chaque semaine de petits changements au lieu d'un trimestre d'un coup → améliore la fréquence de déploiement et réduit la taille du risque à chaque fois. 3) Préparer un retour arrière automatisé et testé (revenir à la version précédente en un clic) → améliore le temps moyen de restauration : on ne répare plus sous pression, on restaure puis on analyse à froid. 4) Supprimer l'arrêt de service : déploiement progressif (rolling ou blue-green) → réduit le délai commit → production, car on n'attend plus une fenêtre du dimanche. Le point commun : aucun de ces changements n'est un « outil », ce sont des pratiques — les outils ne viennent qu'après.

Leçon 2 — Intégration continue : automatiser tests et constructions

L'intégration continue (CI, Continuous Integration) est la pratique qui consiste à intégrer le code de toute l'équipe dans la branche principale très souvent — idéalement chaque jour — et à vérifier chaque intégration par une construction et des tests automatisés. Elle répond à un cauchemar connu : le « vendredi de l'intégration », où l'on découvre que le travail d'Awa et celui de Moussa, corrects séparément, se contredisent une fois réunis. Intégrer tôt et souvent transforme un énorme problème mensuel en minuscules problèmes quotidiens, corrigés en quelques minutes pendant que le contexte est encore frais.

Le cœur de la CI est le pipeline : un fichier de configuration, versionné dans le dépôt avec le code, qui décrit les étapes à exécuter à chaque commit ou demande de fusion : récupérer le code, installer les dépendances, analyser le code (lint), lancer les tests unitaires, puis construire l'artefact (paquet, image Docker). Si une étape échoue, le pipeline est « rouge » : on ne fusionne pas et on ne déploie rien tant qu'il n'est pas redevenu « vert ». Cette règle simple — la branche principale doit toujours être en état de fonctionner — est la condition de tout le reste du cours.

Un bon pipeline est rapide : on vise moins de dix minutes, sinon les développeurs cessent d'attendre son verdict et empilent les commits. On met donc les tests rapides en premier (échec précoce), on met en cache les dépendances téléchargées, et on réserve les tests lents (intégration, bout en bout) à une étape séparée. Enfin, le pipeline ne doit contenir aucun secret en clair : les mots de passe et jetons sont fournis par des variables protégées et masquées de la plateforme, injectées au moment de l'exécution et jamais affichées dans les journaux.

L'essentiel
CI = intégrer souvent dans la branche principale + un pipeline automatique (construire, analyser, tester, fabriquer l'artefact) à chaque commit. Pipeline rouge = on corrige avant tout, on ne fusionne pas, on ne déploie pas. Aucun secret en clair : variables protégées et masquées.

Exemple — Le pipeline d'intégration continue de BiblioAPI (syntaxe générique)

```
# .github/workflows/ci.yml - integration continue de BiblioAPI
name: CI BiblioAPI

on:
push:
branches: [main]
pull_request:
branches: [main]

jobs:
tests:
runs-on: ubuntu-latest
steps:
- name: Recuperer le code
uses: actions/checkout@v4
- name: Installer Python
uses: actions/setup-python@v5
with:
python-version: "3.12"
- name: Installer les dependances
run: pip install -r requirements.txt
- name: Analyser le code
run: flake8 app/
- name: Lancer les tests
run: pytest --cov=app tests/
- name: Construire l'image Docker
run: docker build -t biblioapi:${{ github.sha }} .
```

À retenir

• La CI vérifie CHAQUE intégration : construction + tests automatisés, en quelques minutes.

• Le pipeline est un fichier versionné dans le dépôt : il évolue avec le code et subit les mêmes revues.

• Pipeline rouge = branche principale protégée : on répare d'abord, le reste attend.

Exercice

Le pipeline de BiblioAPI met 47 minutes : les tests unitaires (3 minutes) sont mélangés à 200 tests d'intégration qui démarrent une vraie base de données, et les dépendances Python sont retéléchargées à chaque exécution. Propose trois optimisations concrètes et explique l'effet de chacune sur le comportement de l'équipe.

✔ Corrigé
1) Séparer les étapes : un premier job « rapide » (analyse + tests unitaires, 3 minutes) donne un verdict presque immédiat ; les tests d'intégration tournent ensuite, seulement si le job rapide est vert → échec précoce, on ne gaspille plus 45 minutes quand une simple erreur de syntaxe casse tout. 2) Mettre en cache les dépendances (le dossier des paquets pip, indexé par le contenu de requirements.txt) : le téléchargement ne se refait que lorsque les dépendances changent → plusieurs minutes gagnées à chaque run. 3) Paralléliser : répartir les tests d'intégration sur plusieurs exécuteurs (par dossier ou par module) → la durée totale se rapproche de celle du plus gros paquet au lieu d'être la somme de tous. Effet sur l'équipe : avec un verdict en moins de 10 minutes, les développeurs attendent le vert avant de passer à autre chose, au lieu d'empiler les commits sur un pipeline dont plus personne ne lit les résultats.

Leçon 3 — Déploiement continu et stratégies de mise en production

Après la CI viennent deux pratiques que l'on confond souvent. La livraison continue (Continuous Delivery) prolonge la CI : chaque version qui passe le pipeline est automatiquement préparée et déployée sur un environnement de préproduction, de sorte qu'elle est toujours PRÊTE à passer en production — mais le passage en production reste une décision explicite, déclenchée par un humain (un bouton, une validation). Le déploiement continu (Continuous Deployment) va un pas plus loin : aucune validation humaine, chaque version verte du pipeline part directement en production. La différence tient en un mot : l'endroit où s'arrête l'automatisation.

Déployer souvent ne veut pas dire déployer brutalement. Le déploiement progressif (rolling update) remplace les instances de l'application une à une : pendant l'opération, anciennes et nouvelles versions coexistent, le service ne s'arrête jamais, mais le retour arrière est aussi progressif. Le blue-green maintient deux environnements complets identiques : le bleu sert la production pendant que le vert reçoit la nouvelle version ; on bascule tout le trafic vers le vert d'un coup — retour arrière quasi instantané en rebasculant vers le bleu — au prix d'une infrastructure doublée pendant l'opération.

La stratégie canary est la plus prudente : on envoie d'abord une petite fraction du trafic réel (5 %) vers la nouvelle version, on observe ses métriques (taux d'erreur, temps de réponse) pendant un moment, puis on augmente progressivement (25 %, 50 %, 100 %) si tout va bien. À la moindre alerte, on rebascule tout le trafic sur l'ancienne version : seuls 5 % des utilisateurs ont été exposés. Son compromis : elle exige une supervision digne de ce nom (leçon 5) et complique les migrations de base de données, qui doivent rester compatibles avec les deux versions pendant toute la durée du déploiement.

L'essentiel
Livraison continue = toujours prêt à déployer, mais un humain valide la production. Déploiement continu = production automatique après un pipeline vert. Rolling : instance par instance. Blue-green : bascule totale entre deux environnements, retour instantané, coût doublé. Canary : exposition progressive mesurée, la plus sûre.

Exemple — Canary pas à pas pour BiblioAPI v1.5, et comparaison des stratégies

```
CANARY DE BIBLIOAPI v1.4 -> v1.5
Etape 1 : 95 % du trafic -> v1.4 | 5 % -> v1.5 (observer 30 min)
Etape 2 : 75 % -> v1.4 | 25 % -> v1.5 (si erreurs stables)
Etape 3 : 50 % -> v1.4 | 50 % -> v1.5
Etape 4 : 100 % -> v1.5, v1.4 arretee
Alerte a une etape ? On rebascule a 100 % v1.4 en quelques secondes.

Strategie | Retour arriere | Cout infra | Risque expose
Rolling | Progressif (lent) | Faible | Toutes les
| | | instances, peu a peu
Blue-green | Instantane | Double | Tout le trafic
| (rebascule) | | d'un coup
Canary | Rapide (rebascule) | Moyen | 5 % du trafic
| | | au depart
```

À retenir

- Livraison continue : l'humain garde le bouton de production. Déploiement continu : le pipeline vert suffit.
- Blue-green = retour arrière le plus rapide, mais double infrastructure pendant la bascule.
- Canary = on expose peu d'utilisateurs, on MESURE, puis on décide : la supervision est indissociable du déploiement.

Exercice

Une banque doit mettre à jour son application de virements, qui touche une base de données dont le schéma change (une colonne est renommée). Quelle stratégie de déploiement recommandes-tu, et pourquoi le canary et le blue-green deviennent-ils délicats ici ? Quelle précaution prendre pour la migration de données ?

✔ Corrigé
Recommandation : un déploiement progressif prudent (canary ou rolling) associé à une migration de base en deux temps. La difficulté : avec le blue-green et le canary, les deux versions de l'application tournent EN MÊME TEMPS sur la même base ; si la nouvelle version renomme une colonne, l'ancienne version, encore en service, ne la trouve plus et plante. La précaution est la migration « expand-contract » : d'abord AJOUTER la nouvelle colonne (l'ancien code l'ignore, le nouveau l'utilise), synchroniser les données, déployer la nouvelle version progressivement, et seulement dans une livraison suivante SUPPRIMER l'ancienne colonne quand plus aucun code ne la lit. Règle générale : pendant un déploiement progressif, le schéma de base doit rester compatible avec la version N et la version N−1 de l'application.

Leçon 4 — Conteneuriser une application avec Docker

Un conteneur empaquette une application avec tout ce qu'il lui faut pour fonctionner (bibliothèques, runtime, configuration) dans une image unique et portable : l'image construite sur le portable du développeur est, bit pour bit, celle qui tournera en production. C'est la réponse industrielle au « ça marchait sur ma machine ». Le Dockerfile est la recette de cette image : une suite d'instructions (FROM l'image de base, COPY des fichiers, RUN des commandes de préparation, CMD du démarrage). Chaque instruction produit une couche ; Docker met les couches en cache et ne reconstruit que celles qui changent.

De ce cache découle la règle d'or d'un bon Dockerfile : placer d'abord ce qui change rarement, ensuite ce qui change souvent. On copie donc le fichier de dépendances et on les installe AVANT de copier le code source : tant que les dépendances ne bougent pas, leur couche est réutilisée et la reconstruction prend des secondes. Autres bonnes pratiques : choisir une image de base légère et épinglée (python:3.12-slim plutôt qu'un vague python:latest), ajouter un fichier .dockerignore pour exclure ce qui ne sert pas (dossier .git, caches, documentation), et — règle de sécurité — ne jamais exécuter le conteneur avec l'utilisateur root : on crée un utilisateur dédié et on bascule dessus avec USER avant le CMD.

Pour faire fonctionner plusieurs conteneurs ensemble (l'API, sa base de données, un cache), Docker Compose décrit la pile complète dans un fichier YAML : les services, leurs ports, leurs variables d'environnement, les volumes qui rendent les données persistantes malgré la disparition des conteneurs, et le réseau interne qui les relie. Deux points de vigilance : un volume nommé est indispensable pour les données d'une base (un conteneur supprimé emporte son système de fichiers, pas ses volumes), et la base dans notre exemple n'est exposée sur aucun port externe : seul le service api peut lui parler, via le réseau interne. Les mots de passe y sont injectés par variable d'environnement, jamais écrits en dur dans le fichier versionné.

L'essentiel
Dockerfile = la recette de l'image, instruction par couche, avec cache : dépendances avant le code source. Image de base légère et épinglée, .dockerignore, jamais de root (USER dédié). Docker Compose = la pile multi-conteneurs en YAML : services, volumes persistants, réseau interne, secrets injectés par variables.

Exemple — Le Dockerfile de BiblioAPI, puis sa pile complète avec Docker Compose

```
# Dockerfile de BiblioAPI : la recette de l'image
FROM python:3.12-slim
WORKDIR /app
# Dependances d'abord : couche en cache si elles ne changent pas
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
# Le code ensuite : il change a chaque commit
COPY app/ ./app/
# Ne jamais executer le conteneur en root
RUN useradd --create-home biblio
USER biblio
EXPOSE 8000
CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8000"]

# docker-compose.yml : la pile complete en une commande
services:
api:
build: .
ports: ["8000:8000"]
environment:
DATABASE_URL: postgresql://biblio:${POSTGRES_PASSWORD}@db:5432/biblio
depends_on:
db: { condition: service_healthy }
networks: [biblio-net]
db:
image: postgres:16-alpine
environment:
POSTGRES_USER: biblio
POSTGRES_PASSWORD: ${POSTGRES_PASSWORD}
POSTGRES_DB: biblio
volumes: ["db-data:/var/lib/postgresql/data"]
healthcheck:
test: ["CMD-SHELL", "pg_isready -U biblio"]
interval: 5s
retries: 10
networks: [biblio-net]
volumes: { db-data: }
networks: { biblio-net: }
```

À retenir

• Ordre d'un Dockerfile : base épinglée, dépendances (stables), code (changeant) — sinon le cache ne sert à rien.

• Un conteneur est jetable ; les données vivent dans des volumes, jamais dans le conteneur.

• Compose relie les services par leur nom (ici « db ») sur un réseau interne : inutile d'exposer la base au public.

Exercice

L'image d'une application web pèse 1,4 Go et met 6 minutes à se reconstruire au moindre changement de code. Son Dockerfile commence par FROM ubuntu (image complète), puis COPY . . , puis l'installation des dépendances, et il tourne en root. Identifie les quatre défauts et propose la version corrigée des choix (base, ordre, exécution).

✔ Corrigé
1) Base trop lourde et non épinglée : ubuntu complet alors que l'application n'a besoin que d'un runtime → choisir une image spécialisée et légère (par exemple python:3.12-slim ou node:20-alpine) et toujours épingler la version, sinon la base change sous nos pieds. 2) COPY . . avant l'installation : n'importe quel fichier modifié (même un README) invalide la couche du code ET force la réinstallation des dépendances → inverser : copier d'abord le seul fichier de dépendances, installer, puis copier le code. C'est ce défaut qui explique les 6 minutes. 3) COPY . . sans .dockerignore embarque le dossier .git, les caches et les fichiers locaux (parfois des secrets !) → ajouter un .dockerignore strict. 4) Exécution en root : une faille dans l'application donne les pleins pouvoirs DANS le conteneur → créer un utilisateur dédié et ajouter USER avant le CMD. Résultat typique : image divisée par trois ou quatre, reconstruction de quelques secondes quand seul le code change.

Leçon 5 — Supervision, journaux et alertes en production

Déployer vite sans voir ce qui se passe ensuite serait irresponsable : la supervision (monitoring) est le système sensoriel de la production. Elle repose sur trois familles de données complémentaires. Les métriques sont des nombres mesurés dans le temps (taux de requêtes, taux d'erreurs, temps de réponse, occupation CPU, mémoire, disque) : peu coûteuses à stocker, parfaites pour les tableaux de bord et les alertes. Les journaux (logs) sont les événements détaillés de l'application — qui a fait quoi, quelle erreur exacte — indispensables pour comprendre une panne, mais volumineux et impossibles à lire en continu. Les traces, enfin, suivent une requête à travers plusieurs services.

Pour que les journaux servent à quelque chose, on les rend structurés (une ligne par événement, au format JSON, avec des champs constants : horodatage, niveau — info, avertissement, erreur — service, identifiant de requête) et on les centralise au lieu de les laisser sur chaque machine. Une règle d'hygiène absolue : jamais de données secrètes ni de mots de passe dans les journaux — un journal se partage, se sauvegarde, se montre à un prestataire ; un secret qui y entre y reste. On journalise les identifiants techniques, jamais les credentials.

Une alerte doit être actionnable : quelqu'un sait quoi faire quand elle sonne. On définit donc des seuils chiffrés assortis d'une durée (par exemple : taux de réponses en erreur 5xx supérieur à 5 % pendant 5 minutes) pour ne pas réveiller l'équipe pour un pic d'une seconde. On hiérarchise : avertissement (à regarder aux heures de bureau) et critique (réponse immédiate). Le piège classique est la fatigue d'alerte : trop d'alertes peu fiables, et l'équipe finit par ignorer la vraie. Chaque alerte qui sonne sans exiger d'action doit être corrigée ou supprimée — une alerte n'est pas un pense-bête, c'est un appel au secours.

L'essentiel
Superviser = métriques (des nombres, pour voir et alerter), journaux structurés et centralisés (pour comprendre), traces (pour suivre une requête). Alerte utile = seuil chiffré + durée + action claire. Trop d'alertes inutiles = plus aucune alerte écoutée. Jamais de secret dans les journaux.

Exemple — Règles d'alerte Prometheus pour BiblioAPI (seuils et durées cohérents)

```
# alertes.yml - regles d'alerte Prometheus pour BiblioAPI
groups:
- name: biblioapi
rules:
- alert: TauxErreursEleve
expr: rate(http_requests_total{status=~"5.."}[5m]) > 0.05
for: 5m
labels:
severite: critique
annotations:
resume: "Plus de 5 % de reponses HTTP 5xx sur BiblioAPI"
- alert: LatenceElevee
expr: histogram_quantile(0.95, rate(http_request_duration_seconds_bucket[5m])) > 1
for: 10m
labels:
severite: avertissement
annotations:
resume: "Le p95 des reponses depasse 1 seconde"
- alert: DisquePresquePlein
expr: node_filesystem_avail_bytes{mountpoint="/"} /
node_filesystem_size_bytes{mountpoint="/"} < 0.15
for: 15m
labels:
severite: avertissement
annotations:
resume: "Moins de 15 % d'espace disque libre"
```

À retenir

- Métriques pour détecter, journaux pour diagnostiquer : les deux sont nécessaires, aucun ne remplace l'autre.
- Un seuil d'alerte inclut toujours une durée (for) : on alerte sur un phénomène, pas sur un pic d'une seconde.

• Chaque alerte doit déboucher sur une action connue ; sinon, on la corrige ou on la supprime.

Exercice

Il est 3 h 12 du matin. L'alerte « TauxErreursEleve » vient de sonner pour BiblioAPI : 8 % de réponses 5xx depuis 6 minutes. Un déploiement canary (v1.5, 25 % du trafic) a commencé à 2 h 58. Décris, dans l'ordre, les quatre premiers gestes de l'astreinte, en précisant ce que chacun cherche à établir.

✔ Corrigé
1) Regarder le tableau de bord en séparant le trafic par version : si les erreurs viennent presque toutes de la v1.5 (25 % du trafic mais ~100 % des erreurs), le suspect est identifié — ce geste établit la CORRÉLATION avec le changement récent. 2) Stopper le saignement avant de comprendre : rebasculer le canary à 100 % sur la v1.4 ; le taux d'erreur doit retomber en quelques minutes — on restaure le service d'abord, on enquêtera ensuite. 3) Lire les journaux structurés de la v1.5 filtrés sur l'identifiant d'erreur dominant : le message exact (par exemple une colonne absente après une migration) donne la CAUSE. 4) Consigner l'incident (horaires, gestes, cause) et créer le ticket correctif ; si la cause est une migration de base incompatible, le correctif suivra la démarche expand-contract de la leçon 3 avant toute nouvelle tentative de déploiement. L'ordre est la leçon : détecter, restaurer, comprendre, consigner — jamais l'inverse.

Leçon 6 — Infrastructure as Code : initiation aux principes

Il reste un étage que nous avons jusqu'ici configuré à la main : l'infrastructure elle-même — les serveurs, les réseaux, la base de données, leurs paramètres. L'Infrastructure as Code (IaC) consiste à décrire cette infrastructure dans des fichiers texte, versionnés dans Git au même titre que le code de l'application. Créer un environnement devient alors une exécution de fichier, pas une séance de clics : le même fichier produit le même environnement à Bamako, sur un serveur de test ou chez un autre fournisseur, aujourd'hui comme dans six mois.

Deux philosophies s'opposent. L'impératif énumère les actions à effectuer (« crée un serveur, installe ce paquet, modifie ce fichier ») : il dépend de l'état de départ et casse si l'état a changé entre-temps. Le déclaratif décrit l'état FINAL souhaité (« je veux 2 instances de l'API, une base PostgreSQL 16, ce réseau ») et laisse l'outil calculer les actions nécessaires pour y parvenir. La grande vertu du déclaratif est l'idempotence : appliquer deux fois la même description ne change rien la seconde fois, puisque l'état souhaité est déjà atteint. Le travail suit toujours le même rituel : modifier le fichier, demander le PLAN (l'outil montre ce qu'il va créer, changer, détruire), le faire relire comme une revue de code, puis APPLIQUER.

L'IaC règle aussi un problème insidieux : la dérive (drift). Quand on corrige un paramètre directement sur un serveur « juste pour dépanner », l'infrastructure réelle s'écarte silencieusement de sa description ; quelques mois plus tard, personne ne sait plus comment l'environnement fonctionne, et le reconstruire devient une archéologie.

Avec l'IaC, toute modification légitime passe par le fichier et par Git : l'historique dit qui a changé quoi, quand et pourquoi. Comme pour le code applicatif, les secrets (mot de passe de la base) ne sont JAMAIS dans les fichiers : ils sont déclarés comme variables sensibles, fournies au moment de l'application et masquées dans les affichages.

L'essentiel
IaC = décrire l'infrastructure dans des fichiers versionnés, et laisser un outil la réaliser. Déclaratif (l'état voulu) plutôt qu'impératif (la liste d'actions), d'où l'idempotence : appliquer deux fois ne change rien. Workflow : modifier, planifier, faire relire, appliquer. La dérive manuelle est l'ennemie ; les secrets restent des variables sensibles hors du dépôt.

Exemple — L'infrastructure de BiblioAPI en code déclaratif (pseudocode fidèle, style Terraform)

```
# main.tf - infrastructure de BiblioAPI decrite en code (extraits)
terraform {
required_version = ">= 1.6"
}

# Le mot de passe arrive de l'exterieur : jamais ecrit ici.
variable "mot_de_passe_bdd" {
type = string
sensitive = true
}

resource "exemple_reseau" "biblio" {
nom = "reseau-biblio"
}

resource "exemple_base" "principale" {
moteur = "postgresql"
version = "16"
reseau = exemple_reseau.biblio.id
}

resource "exemple_serveur" "api" {
nom = "biblioapi-prod"
image = "biblioapi:1.5.0"
reseau = exemple_reseau.biblio.id
replicas = 2
variables = {
DATABASE_URL = "postgresql://biblio:${var.mot_de_passe_bdd}@db:5432/biblio"
}
}

output "url_publique" {
value = exemple_serveur.api.url
}
```

À retenir

• Si ce n'est pas dans un fichier versionné, ça n'existe pas : un clic manuel est une dérive en gestation.

• Déclaratif + idempotent : décrire l'état voulu, l'outil trouve les actions ; réappliquer est sans danger.

• Le plan se relit comme du code : on ne laisse jamais un outil « créer, modifier, DÉTRUIRE » sans avoir lu quoi.

Exercice

Sur le serveur de production, un technicien a « juste augmenté la mémoire du conteneur de la base » directement dans l'interface d'administration, sans toucher au fichier d'infrastructure. Trois mois plus tard, l'équipe relance l'application du fichier pour ajouter un serveur — et la mémoire de la base redescend brutalement, provoquant un incident. Explique ce qui s'est passé et la procédure qui aurait évité l'incident.

✔ Corrigé

C'est la dérive : l'état réel (mémoire augmentée à la main) ne correspondait plus à l'état décrit dans le fichier. En appliquant le fichier pour un autre besoin, l'outil a fait converger la réalité vers la description — donc annulé la correction manuelle qu'il ne connaissait pas. L'incident n'est pas la faute de l'outil : il a fait exactement ce que le fichier disait. Procédure correcte : toute modification, même urgente, se fait dans le fichier d'infrastructure, passe par une revue (même rapide) et un plan, PUIS est appliquée ; en urgence absolue, on agit à la main si nécessaire, mais on reporte immédiatement le changement dans le fichier et on le signale à l'équipe. Complément utile : exécuter régulièrement le plan « à vide » pour détecter la dérive avant qu'elle ne morde.

Leçon 7 — Projet : mettre en place un pipeline complet de déploiement

Il est temps d'assembler toutes les pièces sur BiblioAPI. L'architecture cible tient en une phrase : un push sur la branche principale déclenche la CI (analyse, tests, construction de l'image) ; si le pipeline est vert, l'image est publiée dans un registre ; le déploiement canary démarre automatiquement, sous la surveillance des alertes de la leçon 5 ; l'infrastructure qui reçoit tout cela est celle décrite par le code de la leçon 6. Chaque leçon du cours correspond à un étage de cette chaîne, et la faiblesse d'un seul étage (tests absents, alertes muettes, infrastructure manuelle) fragilise l'ensemble.

Deux décisions d'organisation méritent d'être explicites. D'abord, le niveau d'automatisation vers la production : pour BiblioAPI, nous choisissons la livraison continue — le job de déploiement attend une validation humaine (l'« environment » production du pipeline), parce que les bibliothécaires veulent choisir leur moment (jamais un vendredi soir avant un week-end d'examens). Ensuite, la politique de retour arrière : toute version déployée doit pouvoir être remplacée par la précédente en quelques minutes, ce qui impose des migrations de base compatibles (expand-contract) et des images étiquetées par le hash du commit, jamais par un

vague « latest » impossible à tracer.

Le résultat final est le fichier de pipeline ci-dessous : c'est celui des leçons 2 et 3, complété par le job de déploiement. Relis-le comme la synthèse du cours : les déclencheurs, l'ordre strict des jobs (le déploiement « needs » les tests), le secret injecté par variable masquée (jamais en clair), l'étiquette d'image traçable. Avec le Dockerfile de la leçon 4, le fichier Compose, les alertes de la leçon 5 et le fichier d'infrastructure de la leçon 6, tu possèdes une chaîne de livraison complète, reproductible et auditable — exactement ce que les TD et le TP vont te faire pratiquer.

L'essentiel
Le pipeline complet = CI verte OU rien ne part ; déploiement canary automatique mais validation humaine pour la production (livraison continue) ; images étiquetées par le hash du commit ; retour arrière toujours possible ; infrastructure décrite en code. La chaîne vaut ce que vaut son maillon le plus faible.

Exemple — Le pipeline complet de BiblioAPI : CI + déploiement canary (fichier final)

```
# pipeline.yml - BiblioAPI : integration ET deploiement continus
name: Pipeline BiblioAPI

on:
push:
branches: [main]
pull_request:
branches: [main]

jobs:
tests:
runs-on: ubuntu-latest
steps:
- uses: actions/checkout@v4
- uses: actions/setup-python@v5
with:
python-version: "3.12"
- run: pip install -r requirements.txt
- run: pytest --cov=app tests/

deploiement:
needs: tests
if: github.ref == 'refs/heads/main'
runs-on: ubuntu-latest
environment: production
steps:
- uses: actions/checkout@v4
- name: Construire et publier l'image
run: |
docker build -t registre.exemple/biblioapi:${{ github.sha }} .
docker push registre.exemple/biblioapi:${{ github.sha }}
- name: Deployer en canary (5 % du trafic)
env:
JETON_DEPLOIEMENT: ${{ secrets.JETON_DEPLOIEMENT }}
run: ./scripts/deployer.sh canary ${{ github.sha }}
```

À retenir

• Ordre inviolable : tests verts → image publiée → déploiement ; un job « needs » le précédent, jamais l'inverse.

• L'étiquette d'image = le hash du commit : on sait toujours exactement quel code tourne en production.

• Le secret du déploiement vit dans les variables masquées de la plateforme, jamais dans le YAML du dépôt.

Exercice

On te présente le pipeline d'une autre équipe : le job de déploiement ne dépend d'aucun test, l'image est étiquetée « latest », le mot de passe du serveur est écrit en clair dans le fichier YAML, et aucun retour arrière n'est prévu. Classe ces quatre défauts du plus grave au moins grave en justifiant l'ordre, puis donne la correction de chacun.

✔ Corrigé

1) Le mot de passe en clair dans le YAML (le plus grave) : le fichier est versionné, donc le secret est copié chez chaque développeur et conservé pour toujours dans l'historique Git — même « supprimé » plus tard, il reste retrouvable. Correction : variable masquée de la plateforme, et ROTATION du mot de passe déjà exposé (il doit être considéré comme compromis). 2) Aucun retour arrière : un échec en production devient un incident long. Correction : conserver les images précédentes étiquetées et un mécanisme de rebascule (canary/blue-green). 3) Déploiement sans dépendance aux tests : on peut livrer un code que la CI a rejeté. Correction : « needs: tests » et interdiction de forcer. 4) L'étiquette « latest » (le moins grave, mais réel) : impossible de savoir quel code tourne, ni de revenir à « la version d'hier ». Correction : étiqueter par le hash du commit. Remarque : l'ordre peut se discuter entre 2 et 3, mais le secret exposé reste premier — c'est le seul défaut qui ne se répare pas simplement en corrigeant le fichier.

Travaux dirigés (TD)

TD 1 — Autopsie d'une mise en production ratée

Vendredi, 17 h 40. Pour « gagner du temps », un développeur copie les fichiers de la nouvelle version directement sur le serveur de production avec un client FTP, depuis son portable. Le script de connexion à la base, qu'il a modifié pour l'occasion, contient le mot de passe de production en clair ; le dossier entier, script compris, a été commité la veille dans le dépôt Git partagé. Aucun test n'a été lancé : « les changements sont mineurs ». À 18 h 05, les utilisateurs signalent des pages d'erreur. Personne ne sait exactement quels fichiers ont été remplacés ; l'ancienne version n'a été conservée nulle part. Le service est rétabli dimanche, par réinstallation complète. Questions : 1) relève au moins cinq fautes distinctes dans ce récit ; 2) pour chacune, cite la pratique du cours qui l'aurait empêchée ; 3) quelles conséquences a le mot de passe commité, même si on le retire du fichier dès lundi ?

✔ Corrigé du TD 1
Fautes relevées : (1) déploiement manuel et non reproductible (FTP, depuis un portable) — un pipeline automatique (leçons 1, 2 et 7) exécute toujours la même procédure, traçable et répétée ; (2) déploiement un vendredi soir, moment où personne n'est disponible — la livraison continue avec validation humaine choisit son moment ; à défaut, on déploie quand l'équipe est là pour surveiller ; (3) aucun test avant la mise en production — la CI impose des tests verts avant toute livraison (règle du pipeline rouge) ; « changements mineurs » est précisément la phrase qui précède les gros incidents ; (4) aucun retour arrière possible : ni version précédente conservée, ni image étiquetée — le déploiement progressif (canary ou blue-green) et les étiquettes par hash de commit garantissent une restauration en minutes ; (5) le mot de passe de production en clair dans un fichier commité — les secrets vivent dans des variables protégées et masquées, jamais dans le dépôt ; (6) aucune supervision préparée : ce sont les utilisateurs qui ont signalé la panne à 18 h 05, vingt-cinq minutes après les faits — des alertes sur le taux d'erreurs 5xx auraient sonné en 5 minutes. Conséquence du mot de passe commité : il est définitivement compromis. L'historique Git conserve chaque version du fichier, chez chaque personne ayant cloné le dépôt ; le « retirer lundi » n'efface rien. La seule correction valide est la rotation : changer ce mot de passe partout, et considérer l'ancien comme public.

TD 2 — Concevoir les alertes d'une plateforme d'inscriptions universitaires

La plateforme d'inscriptions de l'IAM reçoit 80 % de son trafic annuel pendant deux semaines de septembre : 3 000 étudiants s'inscrivent, paient en ligne et téléchargent leur carte. Le reste de l'année, elle sert quelques centaines de visites par jour. L'équipe veut « être prévenue avant les étudiants quand ça casse, mais pas réveillée pour rien ». Questions : 1) propose quatre métriques à surveiller en priorité et justifie chacune ; 2) pour deux d'entre elles, propose un seuil chiffré et une durée, en distinguant avertissement et critique, et tiens compte du calendrier (pics de septembre contre creux de mars) ; 3) explique pourquoi un seuil fixe de « plus de 100 requêtes par minute = anomalie » serait une mauvaise alerte sur ce service.

✔ Corrigé du TD 2
1) Métriques prioritaires : le taux de réponses en erreur 5xx (le service répond-il juste ?) ; le temps de réponse p95 (le service répond-il vite, notamment sur le tunnel d'inscription et de paiement ?) ; la saturation des ressources (CPU, mémoire, disque de la base) qui annonce les pannes avant qu'elles ne surviennent ; le taux de succès des paiements, métrique MÉTIER : un paiement qui échoue sans erreur technique visible (par exemple un prestataire en panne) ne se voit que là. 2) Exemples de seuils : taux de 5xx supérieur à 2 % pendant 5 minutes → critique (en septembre, chaque minute perdue bloque des files d'étudiants) ; p95 supérieur à 2 secondes pendant 10 minutes → avertissement en période creuse, mais critique pendant les deux semaines d'inscription : les seuils se règlent par période, ou de façon relative (dégradation par rapport à la même heure la veille). Paiements : taux de succès inférieur à 90 % sur 30 minutes → critique à toute saison, car même en mars un étudiant qui ne peut pas payer est bloqué. 3) Le seuil de 100 requêtes/minute est mauvais car il ignore le contexte : en septembre, 500 requêtes/minute sont NORMALES (l'alerte sonnerait en permanence → fatigue d'alerte) ; en mars, 100 requêtes/minute seraient au contraire un événement étrange. Un bon indicateur mesure la SANTÉ (erreurs, latences, succès des opérations), pas le volume brut ; le volume sert à interpréter, pas à alerter.

Travaux pratiques (TP)

TP 1 — Assembler le pipeline complet de BiblioAPI (projet du cours)

En binôme, assemblez la chaîne de livraison complète de BiblioAPI dans un dépôt Git neuf. Livrables attendus : 1) le code minimal d'une API (un point d'entrée, un fichier requirements.txt, au moins huit tests automatisés dont deux qui doivent pouvoir échouer si on casse le code) ; 2) le fichier de pipeline : CI sur chaque demande de fusion (analyse + tests + construction de l'image) et job de déploiement dépendant des tests, avec un secret lu depuis une variable masquée — jamais en clair ; 3) le Dockerfile (base épinglée et légère, dépendances avant le code, utilisateur non-root) et le fichier docker-compose.yml (API + base, volume nommé, base non exposée publiquement) ; 4) un fichier d'alertes avec deux règles à seuils et durées cohérents, et un fichier d'infrastructure déclarative décrivant l'environnement ; 5) un README qui explique comment déployer et comment revenir en arrière. Démonstration finale : deux commits successifs — l'un vert qui se déploie, l'autre volontairement fautif qui doit être bloqué par le pipeline.

✔ Attendus du TP 1
Attendus et critères de réussite : le dépôt contient exactement les pièces pratiquées dans les leçons (pipeline de la leçon 7, Dockerfile et Compose de la leçon 4, alertes de la leçon 5, infrastructure de la leçon 6). Vérifications de l'encadrant : (a) pousser un commit fautif → le pipeline passe au rouge et AUCUN déploiement n'a lieu ; (b) pousser le correctif → le pipeline redevient vert et le déploiement part ; (c) grep du dépôt : aucun mot de passe ni jeton en clair (seul ${{ secrets.... }} ou ${VARIABLE} est accepté) ; (d) le Dockerfile respecte l'ordre base → dépendances → code et contient une instruction USER non-root ; (e) dans le Compose, le service de base n'a pas de clé « ports » publique et ses données sont sur un volume nommé ; (f) le README décrit un retour arrière réaliste (rebasculer sur l'image étiquetée par le hash du commit précédent). Barème indicatif : pipeline fonctionnel et bloquant 30 %, conteneurisation 25 %, supervision et infrastructure 20 %, hygiène des secrets et README 25 %. Erreurs éliminatoires : secret en clair dans le dépôt, déploiement possible malgré des tests rouges.

Exercices corrigés supplémentaires

Exercice 1

Voici un Dockerfile défectueux : « FROM ubuntu / WORKDIR /app / COPY . . / RUN apt-get update et installer python et pip / RUN pip install -r requirements.txt / ENV MOT_DE_PASSE_BDD=soleil123 / CMD python app.py ». Liste ses défauts (il y en a au moins cinq) et réécris-le proprement dans le style de la leçon 4.

✔ Corrigé
Défauts : 1) base ubuntu complète, lourde et non épinglée, alors qu'une image Python slim suffit ; 2) COPY . . placé avant l'installation des dépendances : le cache est invalidé au moindre fichier touché, et tout le dépôt part dans l'image ; 3) pas de .dockerignore : le dossier .git et les fichiers locaux sont embarqués ; 4) le mot de passe de la base est écrit en clair dans l'image (ENV) : quiconque récupère l'image lit le secret — il doit être injecté au moment de l'exécution ; 5) exécution en root (aucun USER) ; 6) CMD en forme shell non structurée. Version corrigée : FROM python:3.12-slim ; WORKDIR /app ; COPY requirements.txt . ; RUN pip install --no-cache-dir -r requirements.txt ; COPY app/ ./app/ ; RUN useradd --create-home biblio ; USER biblio ; EXPOSE 8000 ; CMD ["python", "-m", "app"] — et le mot de passe fourni par variable d'environnement au docker run ou dans le Compose, jamais dans l'image.

Exercice 2

Tableau de bord d'une boutique en ligne, semaine écoulée : taux d'erreurs 5xx passé de 0,3 % (habituel) à 4,8 % mardi entre 14 h et 15 h, puis retour à la normale ; latence p95 stable à 400 ms ; espace disque libre en baisse régulière de 2 % par jour, à 22 % restants dimanche ; le déploiement de mardi 13 h 45 était un rolling update ordinaire. Questions : 1) quelles alertes de la leçon 5 auraient sonné, et lesquelles non ? 2) quel lien fais-tu entre le pic d'erreurs et le déploiement, et que vérifier d'abord ? 3) le disque ne déclenche aucune alerte aujourd'hui : dans combien de jours devient-il critique, et que proposes-tu ?

✔ Corrigé
1) « TauxErreursEleve » (seuil 5 % pendant 5 minutes) : NON, de justesse — 4,8 % reste sous le seuil ; c'est un cas limite qui doit faire réfléchir au réglage : un passage de 0,3 % à 4,8 % est une dégradation massive que le seuil absolu a manquée ; un seuil relatif (par exemple « plus de 5 fois le taux habituel ») l'aurait détectée. « LatenceElevee » : non, p95 stable. « DisquePresquePlein » (moins de 15 %) : pas encore. 2) Le pic démarre un quart d'heure après le rolling update : corrélation forte. Pendant un rolling, anciennes et nouvelles versions coexistent ; un pic temporaire peut venir d'instances nouvelles défaillantes retirées ensuite, ou d'une erreur de la nouvelle version corrigée par le redémarrage. À vérifier d'abord : les journaux par version pendant la fenêtre 14 h – 15 h, et si la cause est dans la nouvelle version, son taux d'erreur propre. 3) À 2 % de baisse par jour depuis 22 %, le seuil de 15 % est atteint dans environ 3 à 4 jours, et un disque de base de données plein est une panne grave. Propositions : alerte d'AVERTISSEMENT dès maintenant sur la TENDANCE (prédiction d'épuisement à 7 jours) plutôt que sur le seul niveau, identifier le consommateur (journaux non purgés ? sauvegardes locales ?) et automatiser la rotation des journaux.

Exercice 3

Un administrateur a noté sa procédure d'installation d'un serveur web : « 1. créer la machine serveur-web-1 ; 2. se connecter en SSH ; 3. installer nginx ; 4. ouvrir le port 443 ; 5. copier le certificat ; 6. redémarrer nginx ». Réécris cette procédure en description déclarative (pseudocode d'état souhaité, style leçon 6), puis explique en trois phrases ce qui change le jour où l'on doit installer un DEUXIÈME serveur identique.

✔ Corrigé
Version déclarative : « serveur web : présent, image = debian-12 ; paquet nginx : installé, version épinglée ; port 443 : ouvert (80 redirigé vers 443) ; certificat : présent, émis par l'autorité X, renouvellement automatique ; service nginx : démarré, activé au boot ». La forme exacte importe moins que le changement de nature : on ne raconte plus une suite de gestes, on décrit l'état que la machine DOIT avoir. Pour le deuxième serveur, la procédure impérative devrait être ré-exécutée à la main, geste par geste, avec le risque d'oublier une étape (en général le certificat ou le port). Avec la description déclarative, on applique le même fichier à une nouvelle cible : l'outil effectue uniquement les actions manquantes, le résultat est garanti identique au premier — et si on ré-applique le fichier au premier serveur, l'idempotence assure qu'il ne se passe rien.

QCM d'évaluation

Choisis la bonne réponse pour chaque question. Les corrigés détaillés se trouvent juste après le QCM : ne les regarde qu'après avoir répondu à tout.

1. L'intégration continue consiste à…

   A) Ne déployer qu'une fois par trimestre, mais très soigneusement

   B) Intégrer le code souvent dans la branche principale, avec construction et tests automatisés à chaque fois

   C) Laisser chaque développeur travailler des mois sur sa branche personnelle

2. En livraison continue, la mise en production…

   A) Reste une décision explicite d'un humain, la version étant toujours prête

   B) Est automatique dès que le pipeline est vert

   C) Est effectuée à la main par copie de fichiers

3. Le principal avantage du blue-green est…

   A) De ne coûter aucune infrastructure supplémentaire

   B) De diviser le trafic en tout petits pourcentages

   C) Un retour arrière quasi instantané, en rebasculant le trafic sur l'environnement précédent

4. Dans un Dockerfile, on copie le fichier de dépendances AVANT le code source pour…

   A) Rendre l'image impossible à copier

   B) Profiter du cache : la couche des dépendances est réutilisée tant qu'elles ne changent pas

   C) Obliger Docker à ignorer le code

5. Un secret (mot de passe, jeton) dans un pipeline doit être…

   A) Écrit en clair dans le YAML, mais seulement en commentaire

   B) Crypté à la main et collé dans le dépôt

   C) Fourni par une variable protégée et masquée de la plateforme, injectée à l'exécution

6. Pour détecter qu'un service se dégrade, on regarde d'abord…

   A) Les métriques (taux d'erreurs, latences, saturation)

   B) La couleur du logo de l'application

   C) Le nombre de lignes du code

7. L'Infrastructure as Code déclarative est précieuse parce qu'elle est…

   A) Réservée aux très grandes entreprises

   B) Écrite impérativement, action par action

   C) Idempotente : appliquer deux fois la même description ne change rien la seconde fois

8. Une stratégie canary consiste à…

   A) Tout arrêter, remplacer, redémarrer

   B) Envoyer d'abord une petite fraction du trafic vers la nouvelle version, mesurer, puis augmenter progressivement

   C) Ne déployer que la nuit

9. Dans un conteneur de production, l'application doit tourner…

A) Avec un utilisateur dédié non privilégié (instruction USER)

B) En root, c'est plus simple pour les fichiers

C) Sans utilisateur, les conteneurs n'en ont pas

10. Le pipeline de déploiement est rouge à l'étape des tests. La bonne conduite est…

A) Forcer le déploiement, les tests « passeront demain »

B) Désactiver les tests gênants

C) Ne rien déployer : corriger le code ou le test, jusqu'au retour du vert

Corrigés du QCM

1. Réponse B — La CI combat le « vendredi de l'intégration » : intégrations fréquentes, vérifiées automatiquement à chaque commit.

2. Réponse A — Livraison continue = toujours prêt + validation humaine. Déploiement automatique sans validation = déploiement continu.

3. Réponse C — Blue-green = deux environnements complets, bascule totale, retour par re-bascule. L'exposition progressive, c'est le canary ; le coût doublé est le prix du blue-green.

4. Réponse B — Chaque instruction forme une couche mise en cache ; installer les dépendances avant le code évite de tout réinstaller à chaque commit.

5. Réponse C — Jamais de secret dans le dépôt : un fichier versionné est copié partout et conservé dans l'historique. Les variables masquées sont injectées au moment du run.

6. Réponse A — Les métriques détectent et alertent ; les journaux servent ensuite à comprendre la cause.

7. Réponse C — Décrire l'état voulu et laisser l'outil converger : c'est l'idempotence, qui rend les environnements reproductibles.

8. Réponse B — Canary = exposition progressive et mesurée : 5 % du trafic, observation, puis montée en charge si les métriques restent saines.

9. Réponse A — Ne jamais exécuter en root : si l'application est compromise, l'attaquant hérite des pleins pouvoirs dans le conteneur.

10. Réponse C — La règle du pipeline rouge est absolue : la branche principale doit toujours être livrable ; déployer malgré le rouge détruit la confiance dans l'outil.

Résumé — les notions importantes

• DevOps : une équipe responsable de bout en bout, des changements petits et fréquents, mesurés par les indicateurs DORA (fréquence, délai, restauration, taux d'échec).

• Intégration continue : chaque commit sur la branche principale déclenche construction, analyse et tests ; pipeline rouge = tout s'arrête jusqu'à correction.

• Livraison continue (prêt + validation humaine) à ne pas confondre avec le déploiement continu (production automatique après un pipeline vert).

• Rolling, blue-green, canary : trois façons de limiter le risque d'une mise en production — coût, vitesse de retour arrière et fraction d'utilisateurs exposés les distinguent.

• Docker : une image par application, construite par un Dockerfile ordonné (base épinglée, dépendances avant code, jamais de root) ; Compose assemble la pile, les données vivent dans des volumes.

• Supervision : métriques pour détecter, journaux structurés et centralisés pour comprendre, alertes à seuils et durées — actionnables ou supprimées ; jamais de secret dans les journaux.

• Infrastructure as Code : déclarative, idempotente, versionnée et relue comme du code ; et partout, en pipeline comme en infrastructure, les secrets restent hors du dépôt, en variables masquées.

Vidéos associées aux leçons

Ces vidéos, en français, complètent les leçons. Liens vérifiés au moment de la publication de ce document.

Leçon 2 — GitLab CI/CD expliqué simplement (Pour développeurs et DevOps)

Chaîne : chillotech · Langue : Français · Niveau : Intermédiaire

Un long tutoriel qui construit un pipeline complet sous GitLab : étapes, jobs, artefacts, variables et passage de la CI à la CD. Parfait pour voir, en vrai, le fichier YAML de la leçon 2 prendre vie sur une plateforme réelle.

https://www.youtube.com/watch?v=mxucWhlawCo

Leçon 2 — Bonnes pratiques (6/6) — Couverture & intégration continue

Chaîne : Bitdyne · Langue : Français · Niveau : Intermédiaire

Pourquoi un pipeline doit pouvoir bloquer une fusion quand un contrôle échoue, et ce que la couverture de tests dit (et ne dit pas) de la qualité. Le complément direct de la leçon 2 sur le rôle de garde-fou de la CI.

https://www.youtube.com/watch?v=ulB13cyoKwc

Leçon 4 — Introduction à Dockerfile et docker compose

Chaîne : Ordirepar · Langue : Français · Niveau : Débutant

La différence entre Dockerfile (la recette d'une image) et Docker Compose (l'orchestration de plusieurs conteneurs), expliquée simplement avant de pratiquer. Idéal avant de refaire à la main les deux fichiers de la leçon 4.

https://www.youtube.com/watch?v=8bRPzDVN4ds

Leçon 4 — Docker & conteneurs : images, Dockerfile, volumes, réseau et Compose – Guide complet

Chaîne : lioncode · Langue : Français · Niveau : Débutant à intermédiaire

Le guide complet : images et couches, écriture d'un Dockerfile efficace, volumes persistants, réseaux entre conteneurs et fichier Compose. Il couvre exactement le

vocabulaire et les gestes de la leçon 4, avec des démonstrations.

https://www.youtube.com/watch?v=YN_nQH6_R-g

Leçon 5 — Superviser ses conteneurs Docker avec Prometheus/Grafana

Chaîne : YoanDev · Langue : Français · Niveau : Intermédiaire

Mise en place réelle d'une supervision de conteneurs : Prometheus collecte les métriques, Grafana les affiche en tableaux de bord. La mise en pratique des métriques et des tableaux de bord de la leçon 5.

https://www.youtube.com/watch?v=_UD181Kf4E0

Sources et références

- Programme officiel Licence Informatique, Université IAM (Institut Africain de Management), Bamako — vérifier les intitulés d'UE auprès du secrétariat pédagogique.
- Kim, Humble, Debois & Willis — « The DevOps Handbook » (IT Revolution Press) : l'ouvrage de référence sur la culture et les pratiques DevOps.
- Humble & Farley — « Continuous Delivery » (Addison-Wesley) : le livre fondateur sur les pipelines de déploiement et la livraison continue.
- Documentation officielle Docker (docs.docker.com) : Dockerfile, bonnes pratiques d'images, Docker Compose et volumes.
- Documentation GitHub Actions (docs.github.com) et documentation Prometheus (prometheus.io) : syntaxe des pipelines, variables masquées, règles d'alerte.
- Forsgren, Humble & Kim — « Accelerate » (IT Revolution Press) : l'étude scientifique derrière les quatre indicateurs DORA.

Fin du cours — Bronbah Tech, Bibliothèque Universitaire IAM. Bon courage pour tes révisions !